

MLA0303_RL_PROGRAM_5.py

```
"""
Experiment 5: Epsilon-Greedy Multi-Armed Bandit for Online Ad Recommendation.
Simulates a stream of user visits where the system must choose which
advertisement to show, balancing exploration (trying ads to learn their
CTR) with exploitation (showing the ad believed to have highest CTR).
"""

import random

# -----
# 1. Bandit setup
# -----
ADS = ["Ad_A", "Ad_B", "Ad_C", "Ad_D"]
TRUE_CTR = {"Ad_A": 0.05, "Ad_B": 0.12, "Ad_C": 0.09, "Ad_D": 0.15} # unknown to agent

EPSILON = 0.10
ROUNDS = 5000

def simulate_click(ad):
    return 1 if random.random() < TRUE_CTR[ad] else 0

# -----
# 2. Epsilon-Greedy algorithm
# -----
def epsilon_greedy():
    Q = {ad: 0.0 for ad in ADS}
    N = {ad: 0 for ad in ADS}
    reward_history = []

    for t in range(1, ROUNDS + 1):
        if random.random() < EPSILON:
            chosen_ad = random.choice(ADS) # explore
        else:
            chosen_ad = max(Q, key=Q.get) # exploit

        reward = simulate_click(chosen_ad)
        N[chosen_ad] += 1
        Q[chosen_ad] += (reward - Q[chosen_ad]) / N[chosen_ad] # incremental update
        reward_history.append(reward)

    return Q, N, reward_history

if __name__ == "__main__":
    random.seed(42)
    Q, N, rewards = epsilon_greedy()

    total_clicks = sum(rewards)
    print(f"Total rounds simulated : {ROUNDS}")
    print(f"Total clicks obtained : {total_clicks}")
    print(f"Overall CTR achieved : {total_clicks / ROUNDS:.4f}\n")

    print(f"{'Ad':<8}{'True CTR':<12}{'Estimated CTR':<16}{'Times Shown'}")
    for ad in ADS:
        print(f"{ad:<8}{TRUE_CTR[ad]:<12}{Q[ad]:<16.4f}{N[ad]}")

    best_ad = max(Q, key=Q.get)
    print(f"\nBest advertisement identified by the agent: {best_ad}")
```

PROGRAM OUTPUT

Total rounds simulated : 5000
Total clicks obtained : 686
Overall CTR achieved : 0.1372

Ad	True CTR	Estimated CTR	Times Shown
Ad_A	0.05	0.0284	211
Ad_B	0.12	0.1152	217
Ad_C	0.09	0.0824	279
Ad_D	0.15	0.1472	4293

Best advertisement identified by the agent: Ad_D